

Advances in Streamflow Forecasting

From Traditional to Modern Approaches

Edited by

Priyanka Sharma and Deepesh Machiwal



Application of different artificial neural network for streamflow forecasting

Md Manjurul Hussain¹, Sheikh Hefzul Bari², Ishtiaq Mahmud³,
Mohammad Istiyak Hossain Siddiquee⁴

¹*Institute of Water and Flood Management, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh;* ²*Department of Civil Engineering, Leading University, Sylhet, Bangladesh;* ³*Civil and Environmental Engineering, Shahjalal University of Science and Technology, Sylhet, Bangladesh;* ⁴*Data and Knowledge Engineering, Otto-von-Guericke University of Magdeburg, Magdeburg, Saxony-Anhalt, Germany*

5.1 Introduction

The term “streamflow” can be referred to as the volume of water flowing through a river or a channel. Streamflow forecasting is a very vital part of water resource management and controlling system. A good forecasting model enables efficient operation of water resources planning and management and includes but not limited to providing water for irrigation and household activities, hydropower planning, drought analysis, flood management, etc. Streamflow is a very complex hydrological process as it is controlled by many climatic, geologic, and physical factors, which make it a challenging task to develop an accurate streamflow forecasting model. Besides, collecting too many variables and putting all these in a model is often cumbersome and inconvenient. In such case, forecasting streamflow using historic flow data has become a convenient option for many researchers and practitioners. Streamflow forecasts, with a lead time of hours or days, are generally used for real-time operation of water resources systems and flood warning purposes. Again, forecasts with lead times of weeks, months, and years are generally used for the water resources planning and management (Govindaraju and Rao, 2013).

There are usually three approaches used for forecasting of a hydrological time series, viz., empirical methods, geomorphological methods, and physical models (ASCE Task Committee on Application of Artificial Neural Networks in Hydrology, 2000). Empirical models are often black-box type owing to their unknown underlying process and complexity. These model developments are

also often region-specific. In the recent past, several empirical models have been developed and proposed for time series forecasting (Salas and Smith, 1981). The second type of method is the geomorphology-based model, which is an improvement over the empirical method (Gupta and Waymire, 1983). The third type of model is the physical-based model, which can be efficiently developed if the physical characteristics of the watershed are known at the model grid scale (ASCE Task Committee on Application of Artificial Neural Networks in Hydrology, 2000). Often, these kinds of data are rarely available in the real world. They also require historical data for model calibration. Therefore, the paradigm of the best model, however, remained unsolved. This leads scientists to seek easy to implement but an efficient alternative solution for forecasting various hydrological processes. In this sense, the artificial neural network (ANN) has become one of the most popular and extensively used models to forecast hydrological processes. The ANNs are inspired by the biological neural networks. According to the above classification, the ANN can be labeled as an empirical model. Like most of the empirical models, ANN is not based on statistical relation. Rather, neurons in ANN learn from experience as seen in the human brain. A detailed discussion about the ANN structure is given later in this chapter.

The ANN technique of forecasting is generally referred to as a black-box model due to its complex and unknown underlying process (Sahoo et al., 2019). It is capable of mapping the nonlinear behavior of hydrological datasets and making predictions without describing the physical characteristics of the process (Kisi and Sanikhani, 2015; Uysal et al., 2016). These characteristics of ANN make the technique well suited for forecasting and modeling hydrological processes.

5.2 Development of neural network technique

The history of the ANN is very old. In the late 1940s, Canadian psychologist Donald O. Hebb (Hebb, 1949) proposed a learning hypothesis based on the mechanism of neuroplasticity. Meanwhile, many researchers were working on the same issue. In 1958, Rosenblatt created an algorithm for pattern recognition called perceptron (Rosenblatt, 1958). Perceptron is mainly single neuron architecture, which is considered as the simplest type of ANN. However, the first multilayer ANN was proposed by Ivakhnenko and Lapa (Ivakhnenko and Lapa, 1966; Schmidhuber, 2015). In 1975, Werbos came with a revolutionary idea named backpropagation algorithm, an excellent error minimization (or learning) algorithm to reduce multilayer neural network error (Werbos, 1975). Kunihiro Fukushima (Fukushima, 1980) introduced “neocognitron,” which later inspired convolutional neural networks (CNNs). Another new type of neural network, Hopfield network, was developed by John Hopfield (Hopfield, 1982), which later contributed to creating one of the most effective networks, the recurrent neural network (RNN).

To date, the progress of the network was mainly confined to the development of several types of networks only. The first breakthrough happened in 1986 when David Rumelhart, Geoff Hinton, and Ronald J. Williams applied the backpropagation algorithm to reduce multilayer network error (Rumelhart et al., 1986). One year later in 1987, the time delay neural network was introduced by Alex Waibel (Waibel, 1987). It used the shift invariance approach; therefore, it can be considered as the first CNN. In 1997, Sepp Hochreiter and Jürgen Schmidhuber invented long short-term memory (LSTM) RNNs, considerably improving the efficiency and practicality of RNNs, which is the most effective network for sequential data (Hochreiter and Schmidhuber, 1997). In the meantime, a lot of improvements were established in learning algorithms (see Almeida et al., 1997; Battiti, 1989; Jacobs, 1988; Lapedes and Farber, 1986; LeCun et al., 1993; Neuneier and Zimmermann, 1998; Silva and Almeida, 1990; Vogl et al., 1988; Yu et al., 1995). Hinton et al. (2006) proposed a new fast learning algorithm that presented how a multi-layered feedforward neural network can be effectively pretrained by one layer at a time, treating each layer in turn as an unsupervised restricted Boltzmann machine (Smolensky, 1986), and then it is fine-tuned using supervised back-propagation. To reduce the overfitting problem in the neural network, the “Dropout Regularization” technique was proposed by Hinton et al. (2012). Another outstanding neural network named generative adversarial network (GAN) was invented by Goodfellow et al. (2014), which is used to generate new data such as images or music.

A lot of different types of neural networks have been invented till today (Fig. 5.1). Most of the neural network models are mainly developed and evolved to solve complex problems like classification and object detection from images or videos, understanding voice commands, etc. Nevertheless, many are also used in time series forecasting (e.g., precipitation, temperature, streamflow, or stock market prediction). Among them, the most commonly used neural network models for time series forecasting are described below.

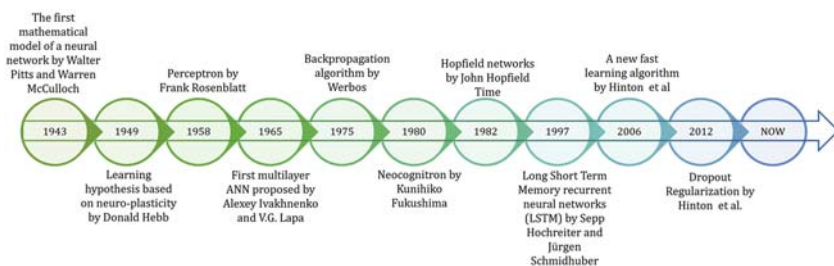


FIGURE 5.1 Timeline of artificial neural network development.

5.2.1 Multilayer perceptron

Multilayer perceptron (MLP), also known as feedforward neural network (FFNN), is the most common neural network architecture used in time series as well as streamflow forecasting (Gupta et al., 2000). Although it is popularly known as MLP, the model is essentially multiple layers of logistic regression models that have continuous nonlinearities rather than perceptrons with discrete values (Bishop, 2006). The MLP consists of three layers, viz., input layer, hidden layer, and output layer. Every layer has one or more neurons, and every neuron is connected with every neuron of the next layer (Fig. 5.2). These types of layers are called fully connected layers or dense layers. The input neurons receive data and multiply with random weights. Weighted data are then passed to the next layer. The very fact that input layer only passes the data without being part of any calculation related to the learning part of the algorithm led many authors to argue about its status as a layer of MLP (Bishop, 2006). However, every neuron of the next layer receives the result, adds bias, and then passes it to an activation function. Activation functions are used to detect nonlinearity in the input–output relationship (Viswanath et al., 2019). Various types of functions can be used as an activation function. The most popular functions are tanh, sigmoid, purelin (linear function), and Rectified Linear Unit (ReLU) (Table 5.1). ReLU is new in comparison to other activation functions, but its popularity has increased rapidly (Géron, 2019; Nair and Hinton, 2010), owing to its faster learning speed and the absence of the vanishing gradient problem (Géron, 2019; Ide and Kurita, 2017). Activation functions help to capture complex relationship between input and output by propagating data from one layer to other layer, which finally reach to the output layer. The results of the output layer are compared with the known target data, which should be the output from the model. The resulting error is then minimized by updating weights using the backpropagation algorithm.

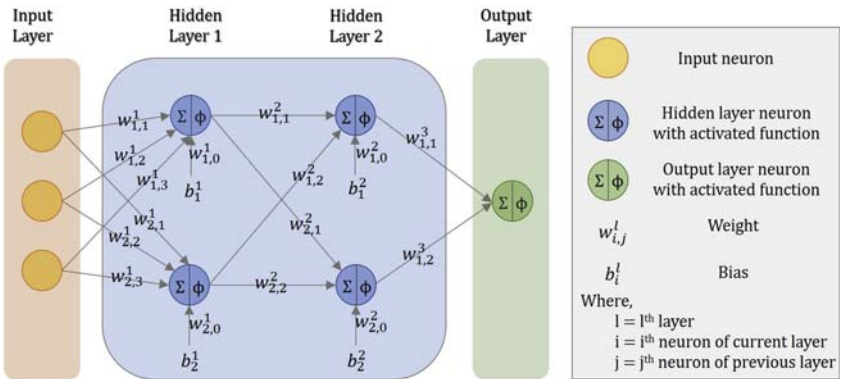
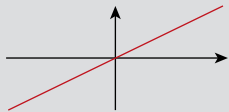
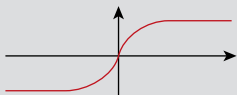
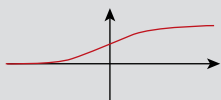
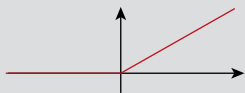
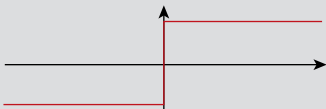


FIGURE 5.2 A typical multilayer network.

TABLE 5.1 List of activation functions.

Activation function	Equation	Graph
Linear	$f(x) = x$	
Hyperbolic tangent (tanh)	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	
Sigmoid (logistic)	$f(x) = \frac{1}{1+e^{-x}}$	
Rectified Linear Unit (ReLU)	$f(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$	
Sign	$f(x) = \begin{cases} -1, & x < 0 \\ 0, & x = 0 \\ 1, & x > 0 \end{cases}$	

Nowadays, a number of variants of backpropagation algorithms, such as gradient descent, momentum optimization, Nesterov Accelerated Gradient, AdaGrad, RMSProp, Adam, Nadam optimization, etc., are available (Géron, 2019). However, Adam optimization is the most popular algorithm used in recent time.

The learning capacity of a neural network depends on the number of neurons and layers. Number of neurons in input and output layers depends on the number of input and output parameters. But there are no rules for selecting number of hidden layers and neurons in those hidden layers. An increase in number of hidden layers or neurons in the hidden layer enhances complexity of the model. A complex neural network model is appropriate to capture the complex relationship between the input and output layers, although it has some

drawbacks also. A complex network model is hard to train and has greater chances to overfit. On the contrary, insufficient number of neurons or hidden layers may make the model underfit. Therefore, it is important to use the appropriate number of neurons and hidden layers to get the best architecture of the model.

Like other machine learning models, the standard practice for modeling an ANN is to split the data into three parts: (1) training dataset, (2) validation dataset, and (3) testing dataset (Hastie et al., 2009). Traditionally, the training dataset consists of 70% of the total dataset, and each 15% of the dataset is used for validation and testing, respectively. The idea is to use training dataset to train the ANN model and validation dataset to keep an eye on the training so that the model does not overfit, whereas testing dataset is used to evaluate the model’s performance in generalizing beyond training data. Numerous goodness-of-fit criteria, namely root mean square error (RMSE), mean square error, Nash–Sutcliffe efficiency etc. are used to evaluate the performance of a model (Géron, 2019). Initially, a number of models using a different combination of neurons and layers are selected. If the performance of a model is good for training dataset and not good for validation dataset, then the model is considered as overfitted. If model’s performance is not good for both training and validation datasets, then the model is assumed to be underfitted. Comparing the results of all the selected models, an optimum model is chosen that performs well for both training and validation datasets (Géron, 2019) (Fig. 5.3).

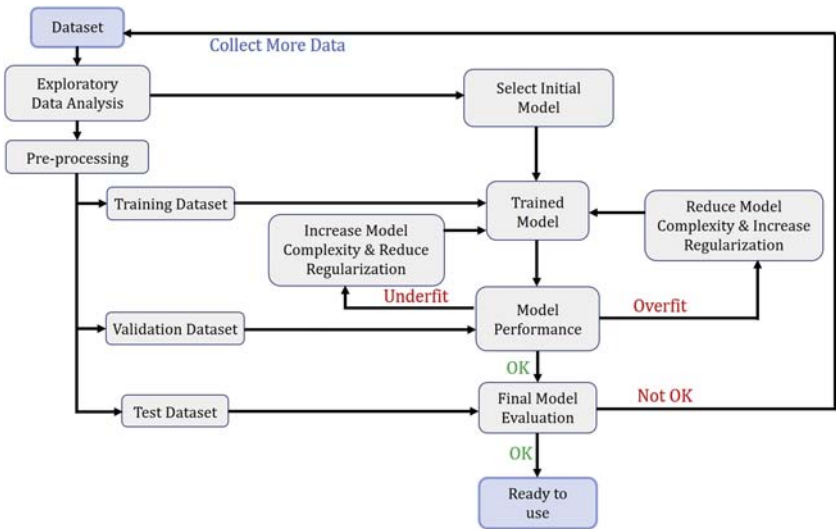


FIGURE 5.3 Application-oriented schematic diagram for machine learning workflow.

5.2.2 Recurrent neural network

One of the major drawbacks of the MLP is that it cannot handle sequential data directly. This problem is resolved using RNNs where data are fed to an RNN model in a sequential manner. Thus, every data in the time series of a parameter depends on the preceding data (Kumar et al., 2019). The RNN works similar to MLP, except that neurons in RNN not only send data to the next layer but also feed data into its own layer (Géron, 2019). Therefore, the preceding data remain present in the layer. Hence, the name is “recurrent” neural network. These self-fed neurons are called memory cell. Due to the model structure, the output of the network depends on both the recent and previous data (Fig. 5.4) (Kumar et al., 2004).

5.2.3 Long short-term memory network

RNN uses backpropagation through time (BPTT) (Werbos, 1990) as the backbone to train the model. BPTT faces serious issues if the sequence is considerably large. Because the gradients calculated through this algorithm need to be passed down from the end of the sequence to the beginning, as this requires a lot of fractional multiplication, the gradient value tends to become smaller as the sequence length increases. This essentially cripples the model by restricting the length of the input sequence. To solve this problem, Hochreiter and Schmidhuber (1997) proposed a new type of RNN, named LSTM. The main difference between the RNN and LSTM is in the memory cell. On inspecting an unrolled LSTM cell, it can be seen that two main

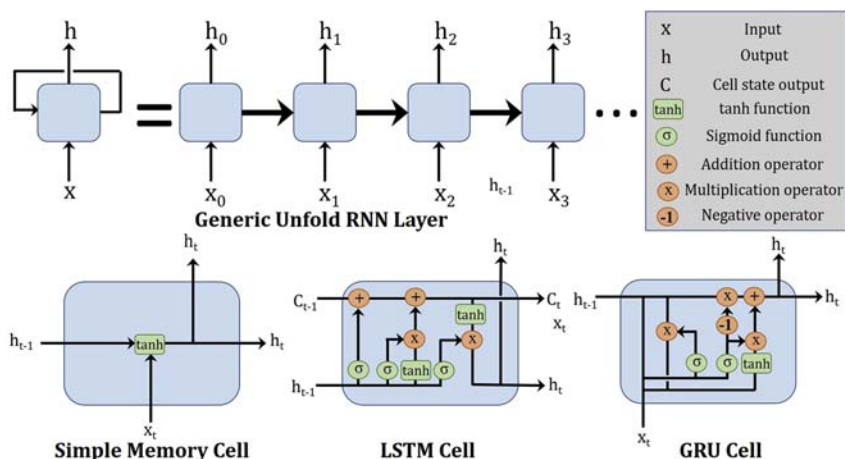


FIGURE 5.4 Different types of memory cell of recurrent neural network. Modified after Olah, C., 2015. Understanding LSTM Networks. Retrieved from: <http://colah.github.io/posts/2015-08-Understanding-LSTMs>.

streams pass cell to cell (Fig. 5.4), where $h(t)$ carries short-term memories and $c(t)$ carries the long-term memories (Géron, 2019). This ensures easy flow of gradients, which in turn allows LSTM to memorize longer sequences as good as shorter sequences.

5.2.4 Gated recurrent unit

The gated recurrent unit (GRU) is a simplified version of the LSTM model. This model, proposed by Cho et al. (2014), mainly combines both short-term and long-term memory streams into a single stream using a gate (Fig. 5.4). Although it is a simpler version of the LSTM, it performs reasonably well (Greff et al., 2016).

5.2.5 Convolutional neural network

CNN is slightly different than other neural network models. CNN is usually used where the spatial arrangement of the data is significant and contains valuable information for the model. It is currently the most widely used network for object (or image) recognition and classification. The CNN consists of two new types of layers: one is the convolution layer and another is the pooling layer (Géron, 2019; Kavitha and Srimathi, 2019; Pratt et al., 2019). These two layers are mainly used for extracting and processing important features from an image (Pratt et al., 2019). The convolution layer uses a set of filter or kernel to slide over the input images. Normally, this filter or kernel is a 2D (for one-layer image, i.e., gray-scale image) or 3D (for multilayer image, i.e., RGB image having three layers) matrix, but 1D kernel is also used for vector data as well as time series data. Besides, there has been other, rather ingenious, proposals of using 1D kernel; for example, as discussed in “Network in Network paper” (Lin et al., 2013), 1D kernel has been used as the fundamental building block for the inception network (Szegedy et al., 2015). This network won the ImageNet challenge in multiple categories. However, after using different types of kernel over the image, results are sent to the pooling layer. The goal of pooling layer is to subsample (i.e., shrink) the input image in order to reduce the computational load, the memory usage, and the number of parameters as well as reduce the risk of overfitting (Géron, 2019). Using the desired number of convolution and pooling layers, the result is sent to a fully connected or dense layer and then to the output layer (Fig. 5.5).

5.2.6 WaveNet

The main idea of WaveNet (Oord et al., 2016) was to design an autoregressive model, in which, statistically speaking, each predictive distribution is conditioned on all previous observations. To achieve this, the authors employed causal convolution technique, whereby part of the input sequence is masked in

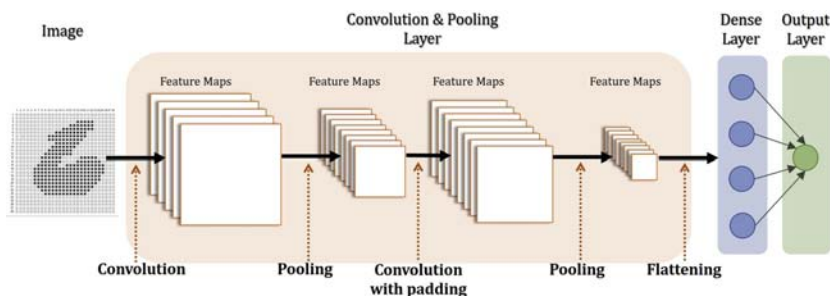


FIGURE 5.5 A typical convolutional neural network architecture. *Modified after Zhou, K., Zheng, Y., Li, B., Dong, W., Zhang, X., 2019. Forecasting different types of convective weather: a deep learning approach. J. Meteorol. Res. 33 (5), 797–809.*

order to blind the input from seeing it. This ensures that the model cannot look ahead in time. Causal convolution has an issue with layer size. In order to increase the receptive field of the model, the required number of layers are much higher than regular CNN. To tackle this challenge, authors used dilated convolution with varying dilation size. Dilated convolution has an added benefit of unchanged dimension of input along the layers, which we cannot have if we use pooling in place of dilation (Géron, 2019; Wan et al., 2019; Wang et al., 2019) (Fig. 5.6). Then, the result of the dilated convolution layer is sent to the dense layer or directly to the output layer as required.

5.3 Artificial neural network in streamflow forecasting

In recent years, ANNs have become popular among engineers and scientists for modeling hydrological processes including streamflow. Karunanithi et al. (1994) used a neural network with cascade correlation algorithm for flow forecasting in the Harun River at the Dexter sampling station in southeast Michigan, USA. They also performed an empirical comparison between neural

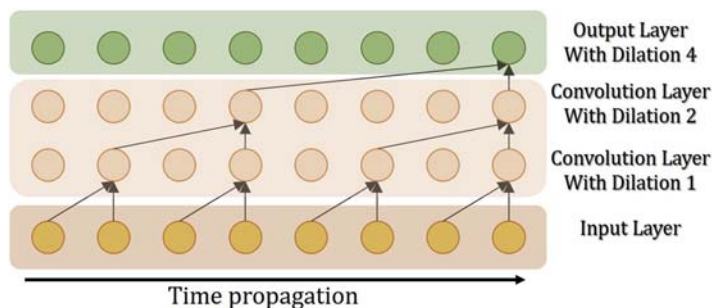


FIGURE 5.6 A typical WaveNet architecture (every circle represents a neuron).

networks and commonly used nonlinear power model. The study found that the result of ANNs was quite encouraging. [Markus et al. \(1995\)](#) forecasted monthly streamflow in the Rio Grande Basin of the United States and compared neural network with periodic transfer function models. The result showed that forecast biases were almost same for both of the methods and RMSE was smaller for neural network. [Poff et al. \(1996\)](#) used the ANN to evaluate the hydrological change of two streams in the northeastern United States. They used historical temperature and precipitation data as input to the ANN. [Muttiah et al. \(1997\)](#) used the cascade correlation algorithm to predict 2-year peak discharge for major river regional basins of the continental United States. They found that the ANN model shows an improvement over the traditional regression techniques. A comparative study among the autoregressive moving average model, the Thomas–Fiering model, and the ANN model using synthetic streamflow data for Pagladiya River in India showed that the ANN performed better than the other two models ([Ahmed and Sarma, 2007](#)). [Kişi \(2009\)](#) found that the neuro-wavelet model significantly increased the accuracy of the ANN model in forecasting daily streamflow in the Ergene River and the Seytan Stream in Turkey. The ANN is also used for the development of streamflow rating curve at the Nile River in Sudan ([Tawfik et al., 1997](#)).

[Xu et al. \(2009\)](#) used ANN and physical-based model TOPMODEL for daily streamflow forecasting in the Baohe River Basin, China. The focus of the study was on evaluating comparative performances of the ANN model and the TOPMODEL. They found that the ANN model performed better than the physical-based model TOPMODEL. A coupled wavelet transform and neural network method was used by [Adamowski and Sun \(2010\)](#) for forecasting flow of the Xeros River at Lazarides and the Kargotis River at Evrythouin Cyprus. Results of the study revealed that the coupled wavelet neural network forecasted streamflow more accurately than the regular ANN model. [Lohani et al. \(2012\)](#) performed a comparative evaluation between autoregressive (AR), ANN, and adaptive neural-based fuzzy inference system (ANFIS) for forecasting of monthly reservoir inflow in the Sutlej River at Bhakra Dam, India. The study reported that the fuzzy inference-based neural model ANFIS provided a comparatively better result than that produced from the ANN and AR models. [Adnan et al. \(2017\)](#) used the ANN and the support vector machine (SVM) to investigate the ability of these two data-driven models for predicting monthly streamflow in the Dainyor hydraulic station in the Indus River in Pakistan. They found that both of the models were useful in predicting monthly streamflow. Several other researchers also used the ANN in streamflow forecasting ([Kişi, 2004, 2005](#); [Yaseen et al., 2015](#); [Uysal et al., 2016](#); [Adnan et al., 2017](#); [Reza et al., 2018](#); [Freire et al., 2019](#); [Rahmani-Rezaeieh et al., 2020](#); [Sun et al., 2019](#)).

It has been seen from the literature that most of the studies forecasted streamflow using MLP-based ANN models. Recently, some other types of

ANN models, such as LSTM (Le et al., 2019; Sahoo et al., 2019), CNN (Pulido-Calvo and Gutierrez-Estrada, 2009; Rehman et al., 2019), etc., are also employed in streamflow forecasting. Yan et al. (2019) used the LSTM and SVM to forecast small watershed streamflow in China and found that the LSTM has a great potential in streamflow forecasting. Moreover, several other ANN models such as GRU and WaveNet are also used recently in time series forecasting (Sit and Demir, 2019; Wan et al., 2019; Wang et al., 2019), but those are rarely used in streamflow forecasting till now.

5.4 Application of ANN: a case study of the Ganges River

A case study is being presented here where relative performance of six different types of neural network models is evaluated by applying the models for forecasting streamflow of the Ganges River. The Ganges, also known as the Ganga (in India) and the Padma (in Bangladesh), is one of the largest river systems in the world. It is running for more than 2500 km from the Himalayas to the Bay of Bengal through India and Bangladesh. In the pathway, Ganges carries $300 \times 10^9 \text{ m}^3$ of water and 520 Mt of sediments annually (Khan et al., 2019). The streamflow discharge data (1969–2018) for the Ganges River recorded at Hardinge Bridge point of Bangladesh were collected from the Bangladesh Water Development Board (BWDB). At first, collected data were converted into weekly streamflow discharge to maintain consistency in time series. The frequency of data was irregular and there were many missing values. Therefore, the number of missing values was counted and presence of inconsistent data was checked (Fig. 5.7). Finally, weekly streamflow data of 18-year period (2001–18) were selected to use in this study. The missing values (about 5%) in the selected dataset were filled up using linear interpolation technique as the ANN models cannot handle missing values. Then the final data were used for model building (Fig. 5.8).

In the univariate (one variable) neural network model, the main assumption is that data are significantly influenced by the past data at certain time lags

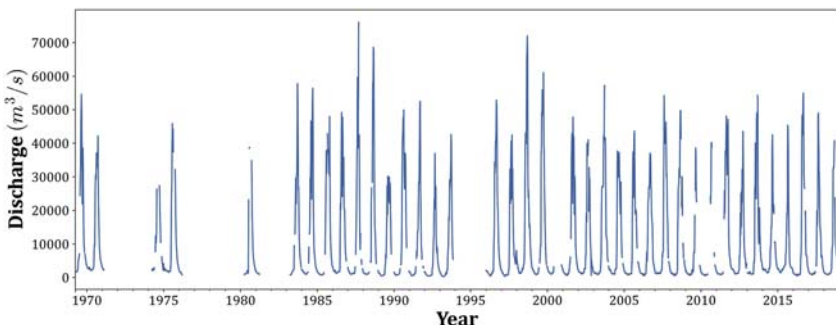


FIGURE 5.7 Temporal variation of Ganges river discharge in Hardinge Bridge point.

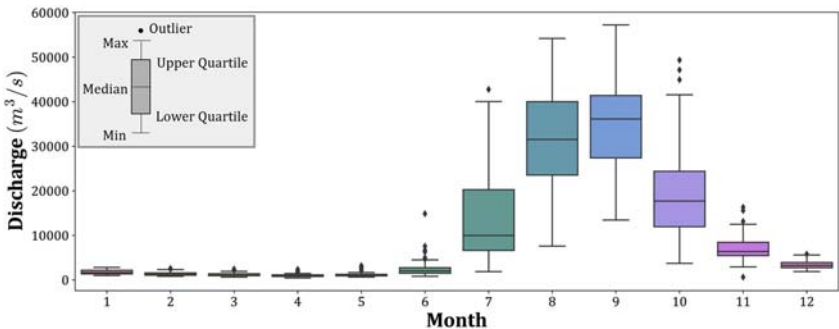


FIGURE 5.8 Monthly boxplot of Ganges river discharge in Hardinge Bridge point.

(Toth and Brath, 2007). Hence, the autocorrelation function (ACF) plot was used to determine the number of significant time lags. The ACF plot (Fig. 5.9) shows that there is a significant correlation in river flow data up to eight lags. Therefore, eight time lags were considered as input in ANN models.

After selecting appropriate time lags, six types of neural network models were developed for streamflow forecasting using training and validation datasets. The models considered in this study are MLP or FFNN, RNN, LSTM, GRU, CNN, and WaveNet. At first, four models (MLP, RNN, LSTM, and GRU) were developed with one hidden layer. The performance of these single hidden layer models was then checked by using different number of neurons (5, 10, 15, 20, 25, and 30) in the hidden layer. Similarly, the performance of the CNN model was checked with 1 and 2 convolutions and pooling layers and the performance of the WaveNet model was checked with 1, 2, 3, 4, and 5 dilated convolutions layer with 2, 4, 8, 16, and 32 dilation rate. Details of the input parameters for the six models along with values of their performance evaluation criteria are summarized in Table 5.2. It can be seen that 20 neurons in hidden layer for MLP, 25 memory cells for RNN, 10 LSTM cells for LSTM

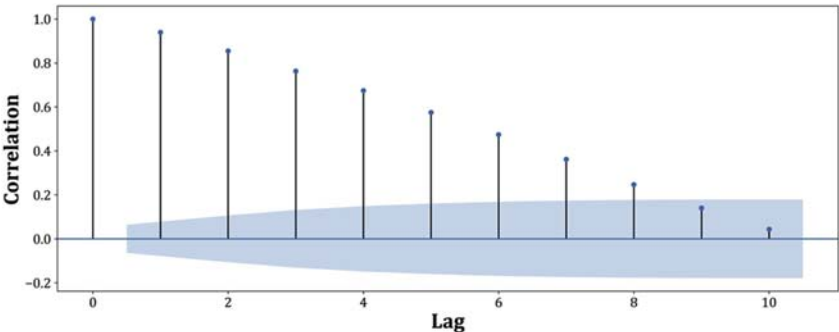


FIGURE 5.9 Autocorrelation function plot for discharge data.

TABLE 5.2 Performance of six different artificial neural network models with different number of neurons in first hidden layer.

Model	Neuron No. in first hidden layer	Convolution layer	Dilation rate	Training set RMSE	Validation set RMSE
MLP	5			5462.229	6042.882
MLP	10			4760.309	4938.453
MLP	15			4516.039	4622.974
MLP^a	20			4538.944	4420.16
MLP	25			4480.836	4589.513
MLP	30			4534.024	4693.206
RNN	5			17,066.69	16,522.93
RNN	10			17,066.69	16,522.93
RNN	15			17,066.69	16,522.93
RNN	20			17,066.69	16,522.93
RNN^a	25			4305.522	4990.427
RNN	30			4384.532	5420.101
LSTM	5			17,260.8	16,659.07
LSTM^a	10			5058.705	5298.559
LSTM	15			17,066.01	16,522.3
LSTM	20			5708.75	6333.627
LSTM	25			7552.782	6790.001
LSTM	30			6533.871	7448.596
GRU	5			11,377.85	10,564.58
GRU	10			17,066.01	16,522.3
GRU	15			4777.475	4789.988
GRU^a	20			4742.919	4713.296
GRU	25			5314.868	5586.754
GRU	30			4676.894	4829.303
CNN^a		1		6644.219	8053.641
CNN		2		6717.886	8032.193
WaveNet		1	2	4488.367	5042.545

Continued

TABLE 5.2 Performance of six different artificial neural network models with different number of neurons in first hidden layer.—cont'd					
Model	Neuron No. in first hidden layer	Convolution layer	Dilation rate	Training set RMSE	Validation set RMSE
WaveNet		2	4	4386.315	5208.171
WaveNet^a		3	8	4384.481	4872.404
WaveNet		4	16	4628.841	5391.81
WaveNet		5	32	4470.203	5063.5
^a Bold lines indicate the best model from each category.					

model, 20 GRU cells for GRU model, 1 convolution layer and pooling layer for CNN, and 3 dilated convolution layers for WaveNet performed the best in each types of model categories. It may also be noted that the CNN model performed the worst, and hence, this model was discarded from the second-step assessment.

In the second step of analysis, a second hidden layer was introduced in each type of model to improve the model performance. In this step, the optimum number of neurons for the first hidden layer was selected based on the first step of analysis. Only number of neurons in the second hidden layer was subjected to trial and error to find out the optimum number of neurons in second hidden layer. This second-step process was completed by following the same procedure that was adopted in the first step of analysis. The only change in the second step was adding one extra dense layer after dilated convolution layers in WaveNet model instead of adding a dilated convolution layer. The number of neurons for this dense layer was calculated as was done previously. The results are shown in Table 5.3. It can be seen from the table that the MLP with 15 neurons in the second hidden layer can be selected as the best-fit model among these six types of ANN models. Therefore, the structure of the final best-fit model is 8–20–15–1 (input–first hidden layer–second hidden layer–output).

The best-fit model was selected based on the training and validation dataset only. Hence, validation of the selected best-fit model is essential using the testing dataset to see how the developed best-fit model performs for the unused dataset. Value of the correlation coefficient between the observed and predicted discharge is found to be more than 93% (Fig. 5.10). Besides, time series plot of observed and predicted data shows a good similarity to each other (Fig. 5.11). Therefore, it is concluded that ANN-based predictions for all training, validation, and testing datasets are almost similar to observed discharge data.

TABLE 5.3 Performance of six different artificial neural network models with different number of neurons in second hidden layer.

Model	Neuron No. in first hidden layer	Training set RMSE	Validation set RMSE
MLP	5	4579.359	4864.14
MLP	10	4526.895	4569.32
MLP^a	15	4509.161	4492.958
MLP	20	4740.721	4567.138
MLP	25	4496.44	4578.662
MLP	30	4556.889	4567.969
RNN	5	4601.588	5711.661
RNN	10	4561.04	5336.098
RNN	15	17,066.69	16,522.93
RNN	20	4479.707	5271.097
RNN	25	4505.346	5259.549
RNN	30	4180.546	5179.065
LSTM	5	17,066.01	16,522.3
LSTM	10	6994.489	7347.332
LSTM	15	7635.621	6757.997
LSTM	20	13,467.3	13,681.73
LSTM	25	17,066.01	16,522.3
LSTM	30	6624.648	7430.409
GRU	5	5332.921	5653.094
GRU	10	17,066.01	16,522.3
GRU	15	6294.408	6777.499
GRU	20	4458.701	4669.333
GRU	25	4532.825	5271.684
GRU	30	4662.329	4739.265
WaveNet	5	4280.906	4899.258
WaveNet	10	4239.503	4816.436
WaveNet	15	4320.778	4803.244
WaveNet	20	4283.443	4869.365

Continued

TABLE 5.3 Performance of six different artificial neural network models with different number of neurons in second hidden layer.—cont'd

Model	Neuron No. in first hidden layer	Training set RMSE	Validation set RMSE
WaveNet	25	4218.217	4879.068
WaveNet	30	4076.18	4818.38

^aBold line indicates the best model.

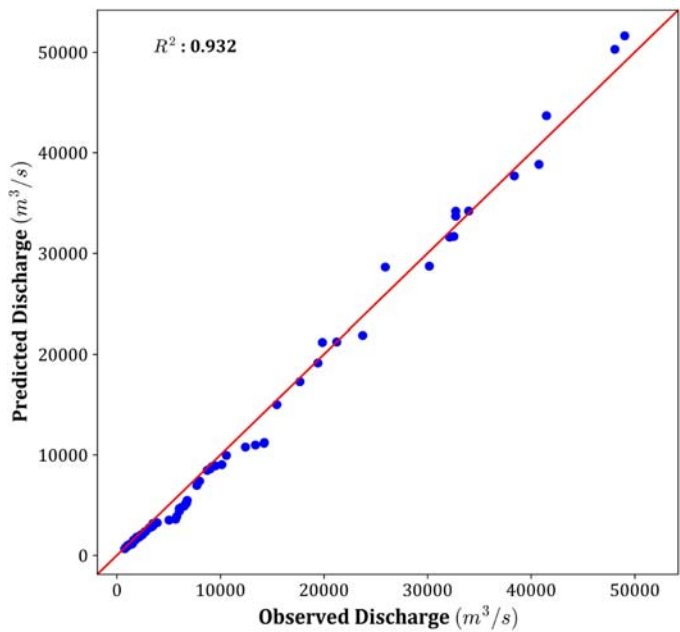


FIGURE 5.10 Q-Q plot of observed and predicted discharge.

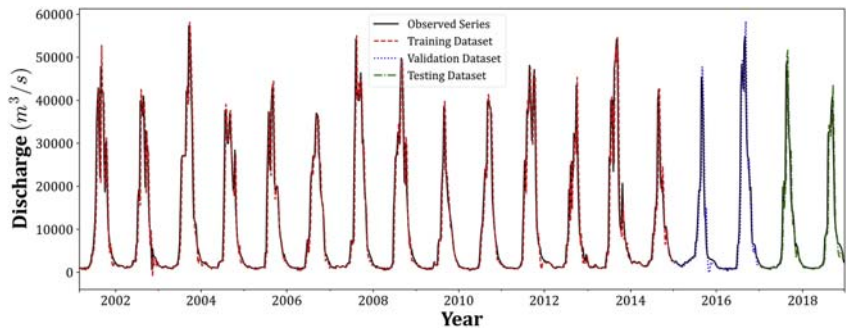


FIGURE 5.11 Comparison of observed and model-predicted streamflow discharge.

5.5 ANN application software and programming language

There are a variety of paid and free tools available for ANN modeling. Matlab (<https://www.mathworks.com/products/matlab.html>) Deep Learning Toolbox (formerly known as Neural Network Toolbox) may be the most used paid software. Flexibility of changing and personalization of code made Matlab one of the most popular. The freely available tools for modeling ANN are Python (<https://www.python.org/>), R (<https://www.r-project.org/>), and Julia (<https://julialang.org/>). Python is one of the most used programming languages to implement ANN modeling. Several python packages for ANN are Keras (<https://keras.io/>), Pytorch (<https://pytorch.org/>), Theano (<http://www.deeplearning.net/software/theano/>), etc. There are several packages available in R (e.g., neuralnet, h2o, nnet, deepnet, rnn, etc.) also. Readers can find the book written by Lewis (2017) interesting for working with time series neural networks in R. Julia is one of the most growing programming languages for data analysis. It also has its own ANN packages such as Flux (<https://fluxml.ai/>), Knet (<https://github.com/denizyuret/Knet.jl>), etc. There are also cross-platform libraries such as TensorFlow (<https://www.tensorflow.org/>) and mxnet (<https://mxnet.apache.org/>). Both are available in multiple programming languages such as python, R, Javascript, Julia, etc. Also, there are several statistical software packages where ANN tools are available (e.g., Statistica, SPSS, etc.).

5.6 Conclusions

ANN models are found performing extremely well in the field of streamflow forecasting. However, there exists further scope to improve the performance of ANN modeling; for example, the efficiency of any ANN-based forecasting model may be substantially improved using multiple input parameters chosen from sensitivity analysis and using hybrid models. Besides, there are many modifications in the existing ANN models, and new algorithms are also developed in recent years. However, despite of improved output, the ANN models cannot provide a clear-cut relationship among interconnected parameters of a hydrological process. From a modeling point of view, a major drawback of the neural networks is that the underlying physical processes or mechanisms are not easily understood whereas statistical or stochastic models can reveal useful information about the series under study.

5.7 Supplementary information

The complete analysis for this chapter can be found at the following link: <https://git.io/JYBtm>.

References

- Adamowski, J., Sun, K., 2010. Development of a coupled wavelet transform and neural network method for flow forecasting of non-perennial rivers in semi-arid watersheds. *J. Hydrol.* 390, 85–91. <https://doi.org/10.1016/j.jhydrol.2010.06.033>.
- Adnan, R.M., Yuan, X., Kişi, O., Yuan, Y., 2017. Streamflow forecasting using artificial neural network and support vector machine models. *Am. Sci. Res. J. Eng. Technol. Sci. (ASRJETS)* 29 (1), 286–294.
- Ahmed, J.A., Sarma, A.K., 2007. Artificial neural network model for synthetic streamflow generation. *Water Resour. Manag.* 21 (6), 1015–1029. <https://doi.org/10.1007/s11269-006-9070-y>.
- Almeida, L.B., Langlois, T., Amaral, J.D., Redol, R.A., 1997. On-Line Step Size Adaptation. INESC, 9 Rua Alves Redol. p. 1000.
- ASCE Task Committee, 2000. Artificial neural networks in hydrology. I: preliminary concepts by the ASCE task committee on application of artificial neural networks in hydrology. *J. Hydrol. Eng.* ASCE 5 (2), 115–123. [https://doi.org/10.1061/\(ASCE\)1084-0699\(2000\)5:2\(115\)](https://doi.org/10.1061/(ASCE)1084-0699(2000)5:2(115)).
- Battiti, R., 1989. Accelerated backpropagation learning: two optimization methods. *Complex Syst.* 3, 331–342.
- Bishop, C.M., 2006. *Pattern Recognition and Machine Learning*. Springer, New York. p. 738.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y., 2014. Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation arXiv:1406.1078 [cs, stat].
- Freire, P.K. de M.M., Santos, C.A.G., da Silva, G.B.L., 2019. Analysis of the use of discrete wavelet transforms coupled with ANN for short-term streamflow forecasting. *Appl. Soft Comput.* 80, 494–505. <https://doi.org/10.1016/j.asoc.2019.04.024>.
- Fukushima, K., 1980. Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol. Cybern.* 36, 193–202. <https://doi.org/10.1007/BF00344251>.
- Géron, A., 2019. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*, 2 edition. O'Reilly Media.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y., 2014. Generative adversarial nets. In: Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N.D., Weinberger, K.Q. (Eds.), *Advances in Neural Information Processing Systems*, vol. 27. Curran Associates, Inc., pp. 2672–2680.
- Govindaraju, R.S., Rao, A.R., 2013. Artificial neural networks in hydrology, vol. 36. Springer Science & Business Media. p. 332. <https://doi.org/10.1007/978-94-015-9341-0>.
- Greff, K., Srivastava, R.K., Koutník, J., Steunebrink, B.R., Schmidhuber, J., 2016. LSTM: a search space odyssey. *IEEE Trans. Neural Netw. Learn. Syst.* 28, 2222–2232. <https://doi.org/10.1109/TNNLS.2016.2582924>.
- Gupta, V.K., Waymire, E., 1983. On the formulation of an analytical approach to hydrologic response and similarity at the basin scale. *J. Hydrol.* 65, 95–123. [https://doi.org/10.1016/0022-1694\(83\)90212-3](https://doi.org/10.1016/0022-1694(83)90212-3).
- Gupta, H.V., Hsu, K., Sorooshian, S., 2000. Effective and efficient modeling for streamflow forecasting. In: Govindaraju, R.S., Rao, A.R. (Eds.), *Artificial Neural Networks in Hydrology*, Water Science and Technology Library. Springer Netherlands, Dordrecht, pp. 7–22. https://doi.org/10.1007/978-94-015-9341-0_2.
- Hastie, T., Tibshirani, R., Friedman, J., 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media. p. 745.

- Hebb, D.O., 1949. *Organization of Behavior*. Wiley, New York. p. 321.
- Hinton, G.E., Osindero, S., Teh, Y.-W., 2006. A fast learning algorithm for deep belief nets. *Neural Comput.* 18, 1527–1554. <https://doi.org/10.1162/neco.2006.18.7.1527>.
- Hinton, G.E., Srivastava, N., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.R., 2012. Improving Neural Networks by Preventing Co-adaptation of Feature Detectors arXiv:1207.0580 [cs].
- Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. *Neural Comput.* 9, 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Hopfield, J.J., 1982. Neural networks and physical systems with emergent collective computational abilities. *Proc. Natl. Acad. Sci. U S A* 79, 2554–2558. <https://doi.org/10.1073/pnas.79.8.2554>.
- Ide, H., Kurita, T., 2017. Improvement of learning for CNN with ReLU activation by sparse regularization. In: 2017 International Joint Conference on Neural Networks (IJCNN). Presented at the 2017 International Joint Conference on Neural Networks. IJCNN, pp. 2684–2691. <https://doi.org/10.1109/IJCNN.2017.7966185>.
- Ivakhnenko, A.G., Lapa, V.G., 1966. *Cybernetic Predicting Devices*, (No. TR-EE66-5). Purdue Univ Lafayette ind School of Electrical Engineering.
- Jacobs, R.A., 1988. Increased rates of convergence through learning rate adaptation. *Neural Network.* 1, 295–307. [https://doi.org/10.1016/0893-6080\(88\)90003-2](https://doi.org/10.1016/0893-6080(88)90003-2).
- Karunanithi, N., Grenney, W.J., Whitley, D., Bovee, K., 1994. Neural networks for river flow prediction. *J. Comput. Civ. Eng.* 8, 201–220. [https://doi.org/10.1061/\(ASCE\)0887-3801\(1994\)8:2\(201\)](https://doi.org/10.1061/(ASCE)0887-3801(1994)8:2(201)).
- Kavitha, B.R., Srimathi, C., 2019. Benchmarking on offline Handwritten Tamil Character Recognition using convolutional neural networks. *J. King Saud Univ. Comput. Inf. Sci.* <https://doi.org/10.1016/j.jksuci.2019.06.004>.
- Khan, M.H.R., Liu, J., Liu, S., Seddique, A.A., Cao, L., Rahman, A., 2019. Clay mineral compositions in surface sediments of the Ganges-Brahmaputra-Meghna river system of Bengal Basin, Bangladesh. *Mar. Geol.* 412, 27–36. <https://doi.org/10.1016/j.margeo.2019.03.007>.
- Kişî, Ö., 2004. river flow modeling using artificial neural networks. *J. Hydrol. Eng. ASCE* 9, 60–63. [https://doi.org/10.1061/\(ASCE\)1084-0699\(2004\)9:1\(60\)](https://doi.org/10.1061/(ASCE)1084-0699(2004)9:1(60)).
- Kişî, Ö., 2005. Daily river flow forecasting using artificial neural networks and auto-regressive models. *Turk. J. Eng. Environ. Sci.* 29, 9–20.
- Kişî, Ö., 2009. Neural networks and wavelet conjunction model for intermittent streamflow forecasting. *J. Hydrol. Eng. ASCE* 14, 773–782. [https://doi.org/10.1061/\(ASCE\)HE.1943-5584.0000053](https://doi.org/10.1061/(ASCE)HE.1943-5584.0000053).
- Kişî, Ö., Sanikhani, H., 2015. Prediction of long-term monthly precipitation using several soft computing methods without climatic data. *Int. J. Climatol.* 35, 4139–4150. <https://doi.org/10.1002/joc.4273>.
- Kumar, D.N., Srinivasa Raju, K., Sathish, T., 2004. River flow forecasting using recurrent neural networks. *Water Resour. Manag.* 18, 143–161. <https://doi.org/10.1023/B:WARM.0000024727.94701.12>.
- Kumar, D., Singh, A., Samui, P., Jha, R.K., 2019. Forecasting monthly precipitation using sequential modelling. *Hydrol. Sci. J.* 64, 690–700. <https://doi.org/10.1080/02626667.2019.1595624>.
- Lapedes, A., Farber, R., 1986. A self-optimizing, nonsymmetrical neural net for content addressable memory and pattern recognition. *Physica D Nonlinear Phenom.* 22, 247–259. [https://doi.org/10.1016/0167-2789\(86\)90244-7](https://doi.org/10.1016/0167-2789(86)90244-7). Proceedings of the Fifth Annual International Conference.
- Le, X.H., Ho, H.V., Lee, G., Jung, S., 2019. Application of long short-term memory (LSTM) neural network for flood forecasting. *Water* 11, 1387. <https://doi.org/10.3390/w11071387>.

- LeCun, Y., Simard, P.Y., Pearlmutter, B., 1993. Automatic learning rate maximization by on-line estimation of the hessian's eigenvectors. In: *Advances in Neural Information Processing Systems*, pp. 156–163.
- Lewis, N.D.C., 2017. *Neural Networks for Time Series Forecasting with R: An Intuitive Step by Step Blueprint for Beginners*. CreateSpace Independent Publishing Platform.
- Lin, M., Chen, Q., Yan, S., 2013. Network in Network arXiv preprint arXiv:1312.4400.
- Lohani, A.K., Kumar, R., Singh, R.D., 2012. Hydrological time series modeling: a comparison between adaptive neuro-fuzzy, neural network and autoregressive techniques. *J. Hydrol.* 442 (443), 23–35. <https://doi.org/10.1016/j.jhydrol.2012.03.031>.
- Markus, M., Salas, J.D., Shin, H.S., 1995. Predicting streamflows based on neural networks. In: *International Water Resources Engineering Conference — Proceedings. Presented at the Proceedings of the 1st International Conference on Water Resources. Part 1 (of 2)*. ASCE, pp. 1641–1646.
- Muttiah, R.S., Srinivasan, R., Allen, P.M., 1997. Prediction of two-year peak stream-discharges using neural Networks1. *J. Am. Water Res. Assoc. (JAWRA)* 33, 625–630. <https://doi.org/10.1111/j.1752-1688.1997.tb03537.x>.
- Nair, V., Hinton, G.E., 2010. Rectified linear units improve restricted Boltzmann machines. In: *Proceedings of the 27th International Conference on Machine Learning. ICML-10*, pp. 807–814.
- Neuneier, R., Zimmermann, H.G., 1998. How to train neural networks. In: Orr, G.B., Müller, K.-R. (Eds.), *Neural Networks: Tricks of the Trade, Lecture Notes in Computer Science*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 373–423. https://doi.org/10.1007/3-540-49430-8_18.
- Olah, C., 2015. Understanding LSTM Networks. Retrieved from. <http://colah.github.io/posts/2015-08-Understanding-LSTMs>.
- Oord, A.V.D., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A., Kavukcuoglu, K., 2016. WaveNet: A Generative Model for Raw Audio arXiv preprint arXiv:1609.03499.
- Poff, N.L., Tokar, S., Johnson, P., 1996. Stream hydrological and ecological responses to climate change assessed with an artificial neural network. *Limnol. Oceanogr.* 41, 857–863. <https://doi.org/10.4319/lo.1996.41.5.0857>.
- Pratt, S., Ochoa, A., Yadav, M., Sheta, A., Eldefrawy, M., 2019. Handwritten digits recognition using convolution neural networks. *J. Comput. Sci. Coll.* 40.
- Pulido-Calvo, I., Gutierrez-Estrada, J.C., 2009. Improved irrigation water demand forecasting using a soft-computing hybrid model. *Biosyst. Eng.* 102, 202–218. <https://doi.org/10.1016/j.biosystemseng.2008.09.032>.
- Rahmani-Rezaeieh, A., Mohammadi, M., Mehr, A.D., 2020. Ensemble gene expression programming: a new approach for evolution of parsimonious streamflow forecasting model. *Theor. Appl. Climatol.* 139 (1–2), 549–564.
- Rehman, S. ur, Yang, Z., Shahid, M., Wei, N., Huang, Y., Waqas, M., Tu, S.-S., Rehman, O. ur, 2019. Water Preservation in Soan River Basin Using Deep Learning Techniques. *CoRR abs/1906.10852*.
- Reza, M., Harun, S., Askari, M., 2018. Streamflow forecasting in bukit merah watershed by using ARIMA and ANN. *Portal: Jurnal Teknik Sipil* 9. <https://doi.org/10.30811/portal.v9i1.612>.
- Rosenblatt, F., 1958. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychol. Rev.* 65, 386–408. <https://doi.org/10.1037/h0042519>.
- Rumelhart, D.E., Hinton, G.E., Williams, R.J., 1986. Learning representations by back-propagating errors. *Nature* 323, 533–536. <https://doi.org/10.1038/323533a0>.

- Sahoo, B.B., Jha, R., Singh, A., Kumar, D., 2019. Long short-term memory (LSTM) recurrent neural network for low-flow hydrological time series forecasting. *Acta Geophys.* 67, 1471–1481. <https://doi.org/10.1007/s11600-019-00330-1>.
- Salas, J.D., Smith, R.A., 1981. Physical basis of stochastic models of annual flows. *Water Resour. Res.* 17, 428–430. <https://doi.org/10.1029/WR017i002p00428>.
- Schmidhuber, J., 2015. Deep learning in neural networks: an overview. *Neural Netw.* 61, 85–117. <https://doi.org/10.1016/j.neunet.2014.09.003>.
- Silva, F.M., Almeida, L.B., 1990. Speeding up backpropagation. In: Eckmiller, R. (Ed.), *Advanced Neural Computers*. North-Holland, Amsterdam, pp. 151–158. <https://doi.org/10.1016/B978-0-444-88400-8.50022-4>.
- Sit, M., Demir, I., 2019. Decentralized Flood Forecasting Using Deep Neural Networks arXiv:1902.02308 [cs, stat].
- Smolensky, P., 1986. Chapter 6: information processing in dynamical systems: foundations of harmony theory. In: Rumelhart, D.E., McClelland, J.L. (Eds.), *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, vol. 1. MIT Press, Foundations, pp. 194–281.
- Sun, Y., Niu, J., Sivakumar, B., 2019. A comparative study of models for short-term streamflow forecasting with emphasis on wavelet-based approach. *Stoch. Environ. Res. Risk Assess.* 1–17.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A., 2015. Going deeper with convolutions. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1–9.
- Tawfik, M., Ibrahim, A., Fahmy, H., 1997. Hysteresis sensitive neural network for modeling rating curves. *J. Comput. Civ. Eng.* 11 (3), 206–211. [https://doi.org/10.1061/\(ASCE\)0887-3801\(1997\)11:3\(206\)](https://doi.org/10.1061/(ASCE)0887-3801(1997)11:3(206)).
- Toth, E., Brath, A., 2007. Multistep ahead streamflow forecasting: role of calibration data in conceptual and neural network modeling. *Water Resour. Res.* 43 <https://doi.org/10.1029/2006WR005383>.
- Uysal, G., Forman, A.A., İensoy, A., 2016. Streamflow forecasting using different neural network models with satellite data for a snow dominated region in Turkey. *Procedia Engineering* 154, 1185–1192. <https://doi.org/10.1016/j.proeng.2016.07.526>.
- Viswanath, S., Saha, M., Mitra, P., Nanjundiah, R.S., 2019. Deep learning based LSTM and SeqToSeq models to detect monsoon spells of India. In: Rodrigues, J.M.F., Cardoso, P.J.S., Monteiro, J., Lam, R., Krzhizhanovskaya, V.V., Lees, M.H., Dongarra, J.J., Sloot, P.M.A. (Eds.), *Computational Science – ICCS 2019, Lecture Notes in Computer Science*. Springer International Publishing, pp. 204–218.
- Vogl, T.P., Mangis, J.K., Rigler, A.K., Zink, W.T., Alkon, D.L., 1988. Accelerating the convergence of the back-propagation method. *Biol. Cybern.* 59, 257–263. <https://doi.org/10.1007/BF00332914>.
- Waibel, A., 1987. Phoneme recognition using time-delay neural networks. In: *Presented at the Meeting of the Institute of Electrical, Information and Communication Engineers (IEICE)*, Tokyo, Japan. <https://doi.org/10.1109/29.21701>.
- Wan, R., Mei, S., Wang, J., Liu, M., Yang, F., 2019. Multivariate temporal convolutional network: a deep neural networks approach for multivariate time series forecasting. *Electronics* 8, 876. <https://doi.org/10.3390/electronics8080876>.
- Wang, J.-H., Lin, G.-F., Chang, M.-J., Huang, I.-H., Chen, Y.-R., 2019. Real-time water-level forecasting using dilated causal convolutional neural networks. *Water Resour. Manag.* 33, 3759–3780. <https://doi.org/10.1007/s11269-019-02342-4>.

- Werbos, P.J., 1975. Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences. Ph. D. dissertation. Harvard University.
- Werbos, P.J., 1990. Backpropagation through time: what it does and how to do it. *Proc. IEEE* 78 (10), 1550–1560. <https://doi.org/10.1109/5.58337>.
- Xu, J., Zhang, W., Zhao, J., 2009. Stream flow forecasting by artificial neural network and TOPMODEL in Baohe River basin. In: 2009 Third International Symposium on Intelligent Information Technology Application Workshops. Presented at the 2009 3rd International Symposium on Intelligent Information Technology Application Workshops (IITAW). IEEE, NanChang, China, pp. 186–189. <https://doi.org/10.1109/IITAW.2009.27>.
- Yan, L., Feng, J., Hang, T., 2019. Small watershed stream-flow forecasting based on LSTM. In: Lee, S., Ismail, R., Choo, H. (Eds.), *Proceedings of the 13th International Conference on Ubiquitous Information Management and Communication (IMCOM) 2019*. Springer International Publishing, Cham, pp. 1006–1014. https://doi.org/10.1007/978-3-030-19063-7_79.
- Yaseen, Z.M., El-Shafie, A., Jaafar, O., Afan, H.A., Sayl, K.N., 2015. Artificial intelligence based models for stream-flow forecasting: 2000–2015. *J. Hydrol.* 530, 829–844. <https://doi.org/10.1016/j.jhydrol.2015.10.038>.
- Yu, X.-H., Chen, G.-A., Cheng, S.-X., 1995. Dynamic learning rate optimization of the back-propagation algorithm. *IEEE Trans. Neural Network.* 6, 669–677. <https://doi.org/10.1109/72.377972>.
- Zhou, K., Zheng, Y., Li, B., Dong, W., Zhang, X., 2019. Forecasting different types of convective weather: a deep learning approach. *J. Meteorol. Res.* 33 (5), 797–809. <https://doi.org/10.1007/s13351-019-8162-6>.